

Scribed by:

1. Shashwat Kasliwal (2022MT11915)
2. Manas Goyal (2022MT11918)
3. Saiyam Jain (2022MT11962)
4. Rohan Roy (2022MT11294)
5. Sarthak Gangwal (2022MT11275)

1 Recap

1.1 Chomsky Normal Form (CNF)

It is not necessary for CFG production rules to be completely unrestricted. A simplified and standardized form called Chomsky Normal Form is sufficient to generate any context-free language.

Theorem: Any context-free language L can be generated by a CFG in Chomsky Normal Form.

Definition: A CFG is in **Chomsky Normal Form (CNF)** if every production rule is in one of the following three forms:

1. $A \rightarrow BC$ (where A, B, C are variables, and B, C are not the start variable)
2. $A \rightarrow a$ (where A is a variable and a is a terminal)
3. $S \rightarrow \lambda$ (where S is the start variable, and this rule is only allowed if $\lambda \in L$)

1.2 Algorithm to Convert a CFG to CNF

Step 1: Introduce a New Start Symbol. Create a new start symbol S_0 and add the rule $S_0 \rightarrow S$, where S was the original start symbol. This ensures the start symbol never appears on the right-hand side (RHS) of a production.

Step 2: Eliminate λ -Productions. Remove all rules of the form $A \rightarrow \lambda$. For every rule where A appears on the RHS, add new rules corresponding to all possible combinations where A is replaced by λ . For example, if we remove $A \rightarrow \lambda$, a rule like $R \rightarrow uAvBw$ would result in a new rule $R \rightarrow uvBw$ being added.

Step 3: Eliminate Unit Productions. Remove all rules of the form $A \rightarrow B$, where both A and B are variables. For each such rule, add new rules $A \rightarrow \alpha$ for every existing rule $B \rightarrow \alpha$. This process is repeated until no unit rules remain.

Step 4: Convert Remaining Rules. Any remaining rules that are not in CNF must be of the form $A \rightarrow X_1X_2 \dots X_k$ where $k > 2$, or they contain terminals on the RHS.

- For rules with terminals on the RHS (e.g., $A \rightarrow aB$), introduce a new variable for each terminal, such as T_a , and add the rule $T_a \rightarrow a$. Then replace the terminal in the original rule: $A \rightarrow T_aB$.
- For long rules with only variables (e.g., $A \rightarrow BCD$), introduce new variables to shorten the rule:

$$\begin{aligned} A &\rightarrow BT_1 \\ T_1 &\rightarrow CD \end{aligned}$$

—

2 Nondeterministic Pushdown Automata (NPDA)

While Finite Automata are sufficient for regular languages, they lack the memory needed for context-free languages (e.g., to recognize $\{0^n1^n\}$). A Pushdown Automaton (PDA) enhances a Nondeterministic Finite Automaton (NFA) with a **stack** memory.

2.1 Nondeterministic vs. Deterministic PDAs

Unlike Finite Automata, where deterministic (DFA) and nondeterministic (NFA) models are equivalent in power, the same is not true for PDAs.

- A **Deterministic Pushdown Automaton (DPDA)** is strictly less powerful than a Nondeterministic Pushdown Automaton (NPDA).
- An **NPDA** is equivalent in power to a CFG. That is, the class of languages recognized by NPDAs is precisely the class of context-free languages.

2.2 Formal Definition of an NPDA

An NPDA is formally defined as a 6-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$, where:

- Q is a finite set of states.
- Σ is a finite set of input symbols (the input alphabet).
- Γ is a finite set of stack symbols (the stack alphabet).
- δ is the transition function, $\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$.

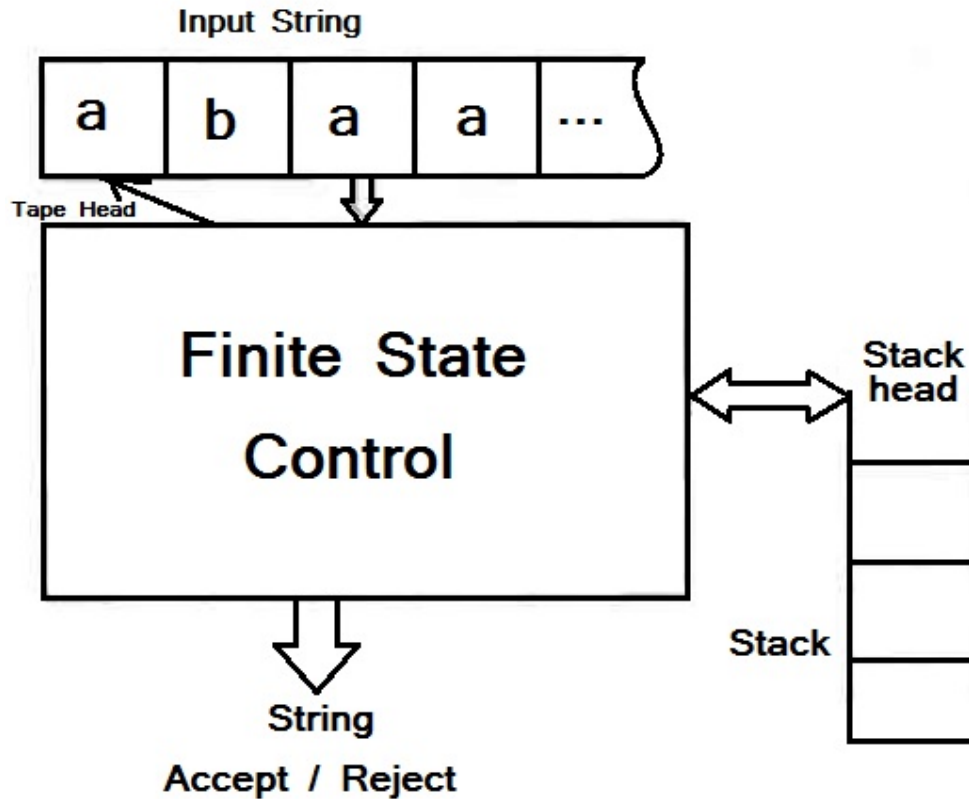


Figure 1: Non deterministic push down automata.

- $q_0 \in Q$ is the start state.
- $F \subseteq Q$ is the set of final (or accepting) states.

2.3 The Transition Function δ

The transition function is the core of the PDA. A transition $(p, \gamma) \in \delta(q, a, X)$ means:

“If the automaton is in state q , the current input symbol is a , and the symbol on top of the stack is X , then it can move to state p and replace X on the stack with the string γ .”

The input symbol a can be λ , allowing the PDA to change state and manipulate the stack without consuming an input symbol.

3 NPDA for $L = \{a^n b^n \mid n \geq 0\}$

We construct a non-deterministic pushdown automaton (NPDA) for the language

$$L = \{a^n b^n \mid n \geq 0\}.$$

3.1 Idea

- For every input symbol a , push a symbol (say 1) onto the stack.
- For every input symbol b , pop a symbol 1 from the stack (if available).
- At the end of input, accept if the stack contains only the special bottom marker.

3.2 Formal Definition

The NPDA M is defined as:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F),$$

where

$$Q = \{q_0, q_1, q_2, q_3\},$$

$$\Sigma = \{a, b\},$$

$$\Gamma = \{1, \$\},$$

q_0 is the start state,

$$F = \{q_0, q_3\}.$$

3.3 Transition Function

The transition function δ is defined as follows:

$$\delta(q_0, \lambda, \lambda) = (q_1, \$),$$

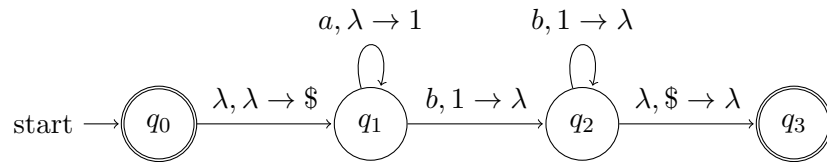
$$\delta(q_1, a, \lambda) = (q_1, 1),$$

$$\delta(q_1, b, 1) = (q_2, \lambda),$$

$$\delta(q_2, b, 1) = (q_2, \lambda),$$

$$\delta(q_2, \lambda, \$) = (q_3, \lambda).$$

3.4 State Diagram



3.5 Explanation

- The automaton starts in q_0 and pushes a special bottom symbol $\$$ to indicate the bottom of the stack.
- In q_1 , for each a read, a symbol 1 is pushed onto the stack.
- Once the automaton begins reading b 's, it transitions to q_2 and pops one 1 for each b .
- If the input is completely read and only $\$$ remains in the stack, the automaton transitions to q_3 and accepts.
- Thus, M accepts exactly the strings of the form $a^n b^n$, including the empty string (since $n = 0$ is allowed).

4 NPDA for $L = \{w \in \{a, b\}^* : n_a(w) = n_b(w)\}$

We construct a non-deterministic pushdown automaton (NPDA) for the language

$$L = \{w \in \{a, b\}^* : n_a(w) = n_b(w)\},$$

i.e., all strings over $\{a, b\}$ having an equal number of a 's and b 's, in any order.

4.1 Idea

- Whenever an a is read, push a counter symbol 0 onto the stack.
- Whenever a b is read, pop a 0 if available, otherwise push a counter symbol 1.
- Whenever an a is read and a 1 is on top of the stack, pop the 1.
- At the end of input, the string is accepted if the stack contains only the bottom marker Z .

4.2 Formal Definition

The NPDA M is given by

$$M = (\{q_0, q_f\}, \{a, b\}, \{0, 1, Z\}, \delta, q_0, \{q_f\}),$$

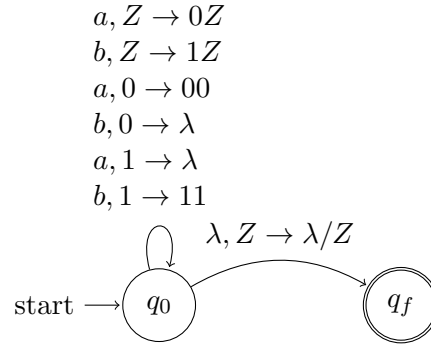
where the stack alphabet $\Gamma = \{0, 1, Z\}$ uses 0 to match a 's, 1 to match b 's, and Z as the bottom-of-stack marker.

4.3 Transition Function

The transition function δ is defined as follows:

$$\begin{aligned}\delta(q_0, \lambda, Z) &= \{(q_f, \lambda)\}, \\ \delta(q_0, a, Z) &= \{(q_0, 0Z)\}, & \delta(q_0, b, Z) &= \{(q_0, 1Z)\}, \\ \delta(q_0, a, 0) &= \{(q_0, 00)\}, & \delta(q_0, b, 0) &= \{(q_0, \lambda)\}, \\ \delta(q_0, a, 1) &= \{(q_0, \lambda)\}, & \delta(q_0, b, 1) &= \{(q_0, 11)\}.\end{aligned}$$

4.4 State Diagram



4.5 Example Derivation

Consider the input string $w = baab$.

$$\begin{aligned}
 (q_0, baab, Z) &\vdash (q_0, aab, 1Z) \\
 &\vdash (q_0, ab, Z) \\
 &\vdash (q_0, b, 0Z) \\
 &\vdash (q_0, \lambda, Z) \\
 &\vdash (q_f, \lambda, Z).
 \end{aligned}$$

Hence the input string $baab$ is accepted.

4.6 Explanation

- Symbols 0 and 1 act as counters for a 's and b 's respectively.
- If the counts of a 's and b 's balance out, the stack returns to Z .
- On empty input with only Z in the stack, the NPDA transitions to q_f and accepts.
- Thus, M accepts exactly the strings where $n_a(w) = n_b(w)$.

5 NPDA for $L = \{ww^R : w \in \{a, b\}^+\}$

We construct a non-deterministic pushdown automaton (NPDA) for the language

$$L = \{ww^R : w \in \{a, b\}^+\},$$

i.e., the set of even-length palindromes over $\{a, b\}$.

5.1 Idea

- For the first half of the string (w), push each symbol onto the stack.
- At some nondeterministically chosen point (the middle), switch states and begin matching the remaining input with the stack contents.
- For the second half (w^R), pop symbols from the stack if they match the current input.
- Accept if the entire input is consumed and only the bottom marker z remains on the stack.

5.2 Formal Definition

The NPDA M is given by

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F),$$

where

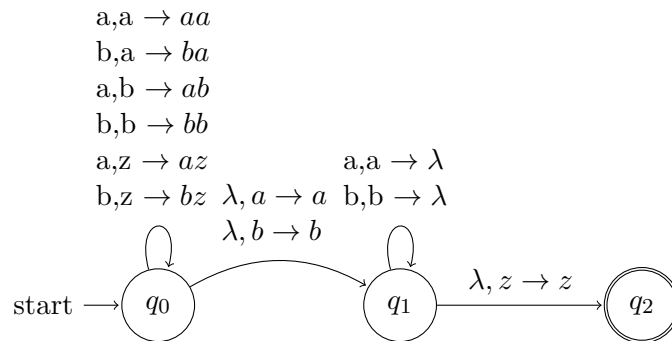
$$Q = \{q_0, q_1, q_2\}, \quad \Sigma = \{a, b\}, \quad \Gamma = \{a, b, z\}, \quad F = \{q_2\}.$$

5.3 Transition Function

The transition function δ is defined as follows:

$$\begin{aligned} \delta(q_0, a, a) &= \{(q_0, aa)\}, & \delta(q_0, b, a) &= \{(q_0, ba)\}, \\ \delta(q_0, a, b) &= \{(q_0, ab)\}, & \delta(q_0, b, b) &= \{(q_0, bb)\}, \\ \delta(q_0, a, z) &= \{(q_0, az)\}, & \delta(q_0, b, z) &= \{(q_0, bz)\}, \\ \delta(q_0, \lambda, a) &= \{(q_1, a)\}, & \delta(q_0, \lambda, b) &= \{(q_1, b)\}, \\ \delta(q_1, a, a) &= \{(q_1, \lambda)\}, & \delta(q_1, b, b) &= \{(q_1, \lambda)\}, \\ \delta(q_1, \lambda, z) &= \{(q_2, z)\}. \end{aligned}$$

5.4 State Diagram



5.5 Example Derivation

For input $abba$:

$$\begin{aligned} (q_0, abba, z) &\vdash (q_0, bba, az) \vdash (q_0, ba, baz) \vdash (q_1, ba, baz) \\ &\vdash (q_1, a, az) \vdash (q_1, \lambda, z) \vdash (q_2, \lambda, z). \end{aligned}$$

Thus $abba \in L$.

5.6 Explanation

- The NPDA nondeterministically decides the midpoint of the string.
- The first half is pushed onto the stack, and the second half is matched against it in reverse order.
- If all symbols match and the stack reduces back to z , the automaton accepts in state q_2 .
- Therefore, M accepts exactly the strings of the form ww^R .

6 NPDA for $L = \{a^i b^j c^k \mid i = j \text{ or } i = k\}$

We construct an NPDA that accepts strings of the form $a^i b^j c^k$ where either $i = j$ or $i = k$. The machine nondeterministically chooses which relation to enforce:

- **Branch 1** (check $i = j$): push one stack symbol for each a , then for each b pop one symbol. After all pushed symbols are popped (stack back to the bottom marker), accept regardless of the number of following c 's.
- **Branch 2** (check $i = k$): push one stack symbol for each a , skip over the b 's (without changing the stack), then for each c pop one symbol. Accept when stack returns to the bottom marker.

6.1 Formal definition

Define

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F),$$

with

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_f\}, \quad \Sigma = \{a, b, c\}, \quad \Gamma = \{1, z\}, \quad F = \{q_f\},$$

where z is the bottom-of-stack marker and '1' is the counter symbol.

6.2 Key transitions

We show the important transitions (set notation suppressed for readability):

$\delta(q_0, \lambda, \lambda) = (q_1, z)$	(push bottom marker and go to reading a 's)
$\delta(q_1, a, \lambda) = (q_1, 1)$	(for each a push a '1')
(nondeterministic choice – choose which equality to check):	
$\delta(q_1, \lambda, \lambda) = (q_2, \lambda)$	(branch 1: begin matching b 's against pushed 1's)
$\delta(q_1, \lambda, \lambda) = (q_3, \lambda)$	(branch 2: skip to b -phase then match c 's)
Branch 1 (check $i = j$) :	
$\delta(q_2, b, 1) = (q_2, \lambda)$	(pop one 1 for each b)
$\delta(q_2, \lambda, z) = (q_f, z)$	(when all 1's popped, accept after reading remaining c 's)
Branch 2 (check $i = k$) :	
$\delta(q_3, b, \gamma) = (q_3, \gamma) \quad (\gamma \in \{1, z\})$	(read/skips any b 's, stack unchanged)
$\delta(q_3, c, 1) = (q_4, \lambda)$	(start popping 1's for each c)
$\delta(q_4, c, 1) = (q_4, \lambda)$	(continue popping for further c 's)
$\delta(q_4, \lambda, z) = (q_f, z)$	(when stack back to z , accept)

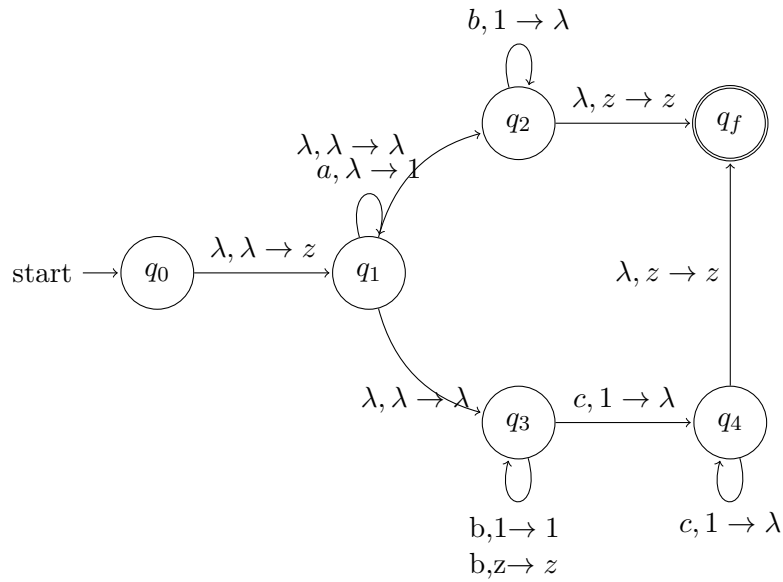
Notes:

- The two λ -moves from q_1 to q_2 and to q_3 are the nondeterministic guesses of which property to enforce.
- In branch 1 we require exactly one popped '1' per b (so $i = j$).
- In branch 2 we skip all b 's then pop one '1' per c (so $i = k$).

6.3 Example derivations

- Input $a^2b^2c^3$ (here $i = j = 2$, branch 1 chosen): push two '1's for the a 's, pop two '1's for the two b 's, reach z , move to q_f and accept (remaining c 's are ignored).
- Input $a^2b^1c^2$ (here $i = k = 2$, branch 2 chosen): push two '1's for a 's, skip the single b , then pop two '1's for the two c 's; reach z and accept.

6.4 State diagram



6.5 Remarks

- The machine is nondeterministic because it must guess whether $i = j$ or $i = k$. A deterministic PDA cannot in general make this guess without extra information.
- The use of the bottom marker z lets us detect when all pushed ‘1’s have been popped.

References

- [1] P. Linz, *An Introduction to Formal Languages and Automata*, Jones & Bartlett Learning, 2012.